
Deep
Learning
School

Описание домашнего задания



В данном задании вы будете работать с датасетом о персонажах из вселенной Игры Престолов A Wiki of Ice and Fire. Вам предстоит предсказать, кто из персонажей умрет, а кто останется вживых.

Описание данных:

- **name**: Имя персонажа
- **Title**: Социальный статус или знатность
- **House**: Дом, к которому принадлежит персонаж
- **Culture**: Социальная группа, к которой принадлежит персонаж
- **book1/2/3/4/5**: Появление персонажа в книге
- **Is noble**: Знатность персонажа, основанное на титуле
- **Age**: Отсчет времени: 305 AC
- **male**: Мужчина или женщина
- **dateOfBirth**: дата рождения
- **Spouse**: Имя супруги\а персонажа
- **Father**: Имя отца персонажа
- **Mother**: Имя матери персонажа
- **Heir**: Имя наследника персонажа
- **Is married**: Represents whether the character is married
- **Is spouse alive**: Represents whether character's spouse is alive
- **Is mother alive:** Жива ли мать персонажа
- **Is heir alive:** Жив ли наследник персонажа
- **Is father alive:** Указывает, жив ли отец персонажа
- **Number dead relations:** Количество умерших персонажей, с которыми персонаж связан
- **Popularity score:** Количество внутренних входящих и исходящих ссылок на страницу персонажей в вики http://awoiaf.westeros.org

Целевая переменная:

- **isAlive**: жив ли персонаж в книге

Оценивание:

Баллы считаются следующим образом:

1) $1.00 \geq score \geq 0.65$ --- 5 баллов

2) $0.65 > score \geq 0.50$ --- 4 балла

3) $0.50 > score \geq 0.45$ --- 3 балла

4) $0.45 > score \geq 0.40$ --- 2 балла

5) $0.40 > score \geq 0.35$ --- 1 балл

6) $0.35 > score$ --- 0 баллов

Часть 1. Анализ и предобработка данных

Здесь вам необходимо сделать все шаги, которые обсуждались в первой части семинара.

- Предобработка данных
  - Обработка пропущенных данных
  - Создание новых признаков
  - Удаление ненужных столбцов
- Анализ данных
  - Анализ целевой переменной
  - Анализ признаков
  - Анализ влияния признаков на целевую переменную
- Подготовка данных для обучения модели

Загружаем датасет

Скачивание в colab

```
# !gdown 1h99toeF7lZ2I3iJwehgKO-QQmDaOe_O3 # test dataset
# !gdown 1XL0VTygpZj-ZAuTNRBgApZTPQyNDnT-v # train dataset
```

Скачивание на локальный пк

```
import os

FOLDER = 'content'

# Проверка наличия папки и создание по необходимости
os.makedirs(FOLDER, exist_ok=True)

import gdown

# Скачивание train

FILE_ID = '1XL0VTygpZj-ZAuTNRBgApZTPQyNDnT-v'
PATH = os.path.join(FOLDER, 'game_of_thrones_train.csv')
```

```
# Проверка наличия файла и скачивание по необходимости
if not os.path.exists(PATH):
    gdown.download(f'https://drive.google.com/uc?id={FILE_ID}', PATH,
quiet=True)
    print('Файл скачан:', PATH)
else:
    print('Файл уже скачан')
```

```
Файл уже скачан
```

```
# Скачивание test

FILE_ID = '1h99toeF7lZ2I3iJwehgKO-QQmDaOe_O3'
PATH = os.path.join(FOLDER, 'game_of_thrones_test.csv')

# Проверка наличия файла и скачивание по необходимости
if not os.path.exists(PATH):
    gdown.download(f'https://drive.google.com/uc?id={FILE_ID}', PATH,
quiet=True)
    print('Файл скачан:', PATH)
else:
    print('Файл уже скачан')
```

```
Файл уже скачан
```

Задание 1.1. Импортируйте библиотеки pandas, matplotlib, seaborn

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn
```

Задание 1.2. Загрузите датасет в Pandas DataFrame при помощи функции `read_csv`. Вместо дефолтных наименований строк `0,1,...`, при помощи параметра `index_col`, сделайте значения колонки `S.No` наименованиями строк:

```
import warnings
warnings.filterwarnings('ignore')

# PATH = '/content/' # colab
PATH = 'content/' # локальный

data = pd.read_csv(PATH + 'game_of_thrones_train.csv',
index_col='S.No')
data.head()
```

```
                        name                  title  male   culture
dateOfBirth  \
S.No

1     Viserys II Targaryen                    NaN     1       NaN
```

```
NaN
2              Walder Frey  Lord of the Crossing     1  Rivermen
208.0
3             Addison Hill                   Ser     1       NaN
NaN
4              Aemma Arryn                 Queen     0       NaN
82.0
5           Sylva Santagar            Greenstone     0   Dornish
276.0

                   mother            father                 heir  \
S.No
1     Rhaenyra Targaryen  Daemon Targaryen  Aegon IV Targaryen
2                     NaN               NaN                 NaN
3                     NaN               NaN                 NaN
4                     NaN               NaN                 NaN
5                     NaN               NaN                 NaN

                 house               spouse  ...  isAliveMother
isAliveFather  \
S.No                                         ...

1                  NaN                  NaN  ...            1.0
0.0
2          House Frey          Perra Royce  ...            NaN
NaN
3        House Swyft                  NaN  ...            NaN
NaN
4        House Arryn  Viserys I Targaryen  ...            NaN
NaN
5     House Santagar      Eldon Estermont  ...            NaN
NaN

      isAliveHeir  isAliveSpouse  isMarried  isNoble   age
numDeadRelations  \
S.No

1             0.0            NaN          0        0   NaN
11
2             NaN            1.0          1        1  97.0
1
3             NaN            NaN          0        1   NaN
0
4             NaN            0.0          1        1  23.0
0
5             NaN            1.0          1        1  29.0
0

      popularity  isAlive
S.No
```

```
1       0.605351        0
2       0.896321        1
3       0.267559        1
4       0.183946        0
5       0.043478        1

[5 rows x 25 columns]
```

Посмотрите, какие типы данных представлены в нашем датасете

Знакомый нам метод describe() возвращает различную информацию для столбцов с числовыми типами данных, и с типами данных *object*

Давайте посмотрим на вывод для типа данных *object*. Для этого:

- сначала применим метод describe(). Укажем в качестве аргумента тип данных столбцов, статистику по которым мы хотим посмотреть (см. https://pandas.pydata.org/docs/reference/api/pandas.Series.map.html)
- для удобства восприятия транспонируем таблицу

```
data.describe(include='object').T
```

```
         count unique                   top freq
name      1557   1557  Viserys II Targaryen    1
title      717    195                   Ser  306
culture    488     51              Northmen   94
mother      18     16    Rhaenyra Targaryen    2
father      22     19      Daemon Targaryen    2
heir        21     20   Jaehaerys Targaryen    2
house     1176    315            House Frey   89
spouse     200    186           Walder Frey    6
```

Теперь давайте посмотрим на столбцы с числовыми типами данных. Дополните код ниже. Для удобства восприятия мы транспонировали таблицу и ограничили вывод тремя столбцами - количество строк без NaN, максимальное и минимальное значение (о кастомизации вариантах вывода describe() вы можете почитать в документации по ссылке выше).

```
data.describe(include='number').T[['count', 'min', 'max']]
```

```
                  count   min    max
male             1557.0   0.0    1.0
dateOfBirth       279.0 -25.0  299.0
book1            1557.0   0.0    1.0
book2            1557.0   0.0    1.0
book3            1557.0   0.0    1.0
book4            1557.0   0.0    1.0
book5            1557.0   0.0    1.0
isAliveMother      18.0   0.0    1.0
isAliveFather      22.0   0.0    1.0
```

```
isAliveHeir          21.0   0.0    1.0
isAliveSpouse       200.0   0.0    1.0
isMarried          1557.0   0.0    1.0
isNoble            1557.0   0.0    1.0
age                 279.0   0.0  100.0
numDeadRelations   1557.0   0.0   15.0
popularity         1557.0   0.0    1.0
isAlive            1557.0   0.0    1.0
```

Так мы проверим, есть ли в данных неадекватнные значения. Большинство числовых столбцов - это числа от 0 до 1. Отрицательные значения `dateOfBirth` не являются ошибкой. Значения age и `numDeadRelations` также выглядят адекватными. Можно переходить к дальнейшим шагам анализа (анонс - а в тестовых данных нас будет ждать сюрприз).

Проверим на наличие дубликатов строк (объектов)

```
data.duplicated().sum()
```

```
0
```

Задание 1.3. Предобработка (очистка) данных.

В нашем домашнем задании все пропуски в данных (missing values) уже закодированы как NaN. Проанализируйте, в каких колонках и как часто встречаются NaN значения. Далее вам надо будет принять решение, как их обрабатывать.

Взглянем на общую длину обучающей выборки

```
len(data)
```

```
1557
```

Теперь посмотрим на количество пропусков в столбцах обучающей выборки

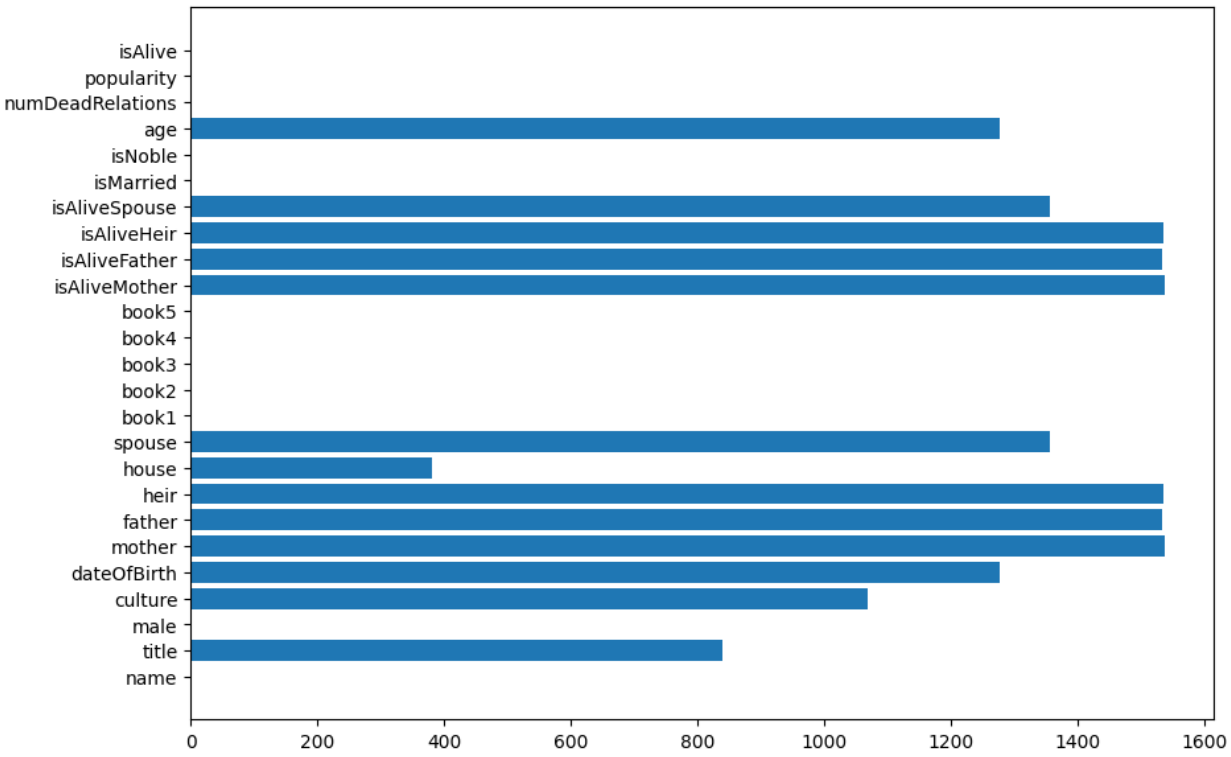
```
cols_with_missing = data.isna().sum()
cols_with_missing[cols_with_missing > 0]
```

```
title             840
culture          1069
dateOfBirth      1278
mother           1539
father           1535
heir             1536
house             381
spouse           1357
isAliveMother    1539
isAliveFather    1535
isAliveHeir      1536
isAliveSpouse    1357
```

```
age                1278
dtype: int64

plt.figure(figsize=(10, 7))
plt.barh(cols_with_missing.index, cols_with_missing.values)

plt.show();
```



В этом задании удалять строки с NaN (dropna) мы не будем по следующим причинам:

- в обучающем датасете много признаков с большим количество пропусков. Если удалять все строки с NaN, то размер выборки сильно уменьшится. Мы потеряем много данных, которые можно было бы использовать для построения более точной модели.
- тестовом датасете также много признаков с NaN (вы можете в этом убедиться, если скачаете датасет и совершите с ним те же действия, что выше проделали с обучаюшим датасетом). Поэтому нам все-равно придется придумать способ кодировать NaN, чтобы модель делала прогнозы для всех персонажей из тестового датасета. Для этого нам потребуется сохранять, а не удалять данные в обучающем датасете.

Как вы могли заметить, в наших данных очень много пропущенных значений, причём в некоторых случая пропущена **большая** часть значений. Поэтому заполнять по умолчанию медианой/средним/модой в данном случае - не самый лучший способ (однако, это довольно часто используемый метод заполнения, который может пригодиться вам в будущем)

Ниже мы посмотрим, как можно работать с признаками с большим количеством пропущенных значений.

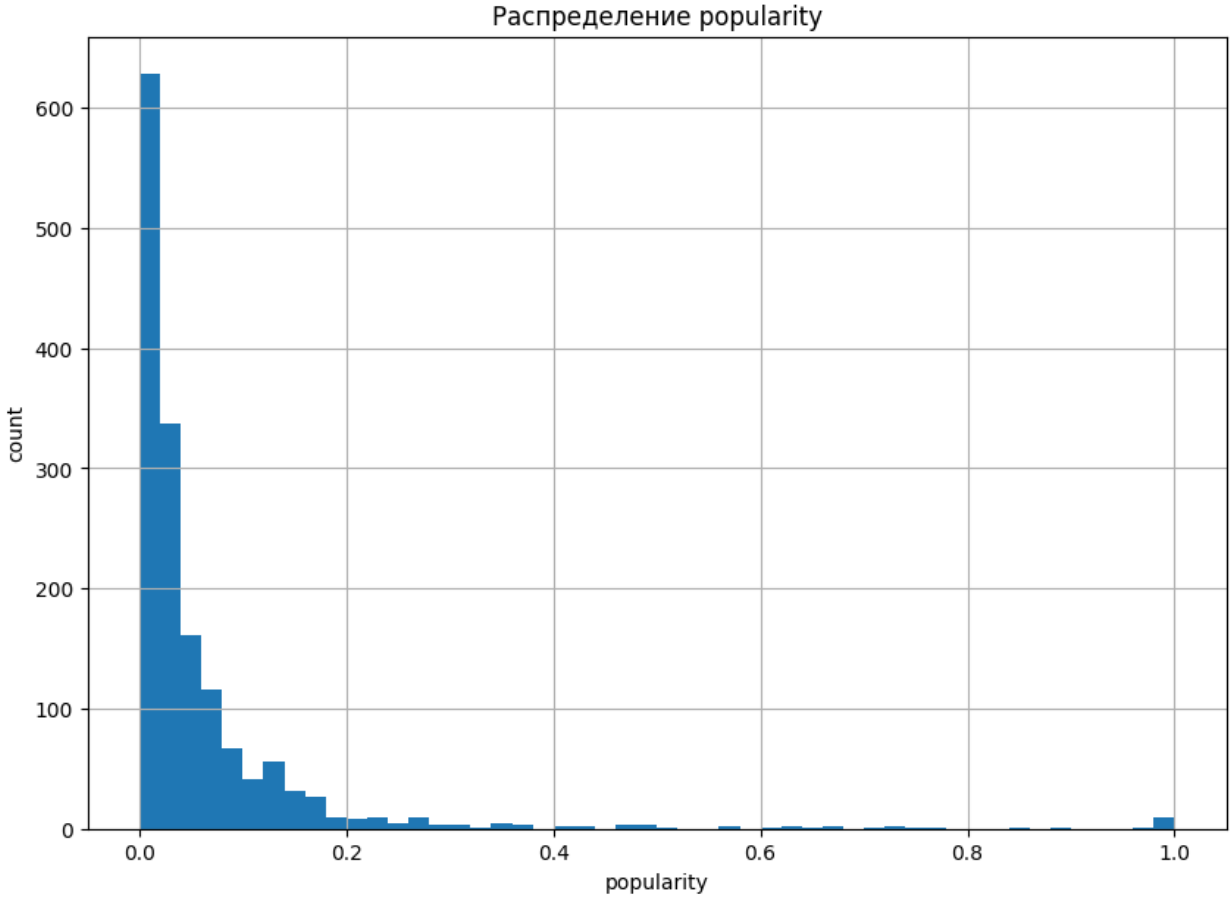
Задание 1.4. Числовые признаки

У нас есть **признак popularity**. Постройте гистограмму распределения данного признака с количеством интервалов (bins), равным 50 (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.hist.html)

```
plt.figure(figsize=(10, 7))

plt.hist(x=data['popularity'], bins=50)
plt.xlabel('popularity')
plt.ylabel('count')
plt.title('Распределение popularity')
plt.grid(True)

plt.show();
```



Распределение сильно несимметрично. Можно преобразовать данный признак, например, по формуле `np.log10(data["popularity"]*M+1)` (добавляем 1 ради логарифма, так как для некоторых персонажей `popularity==0`). В качестве M можно попробовать, например, M=100 или другое число.

При желании для `popularity` вы можете использовать свой способ шкалирования признаков с несимметричным распределением.

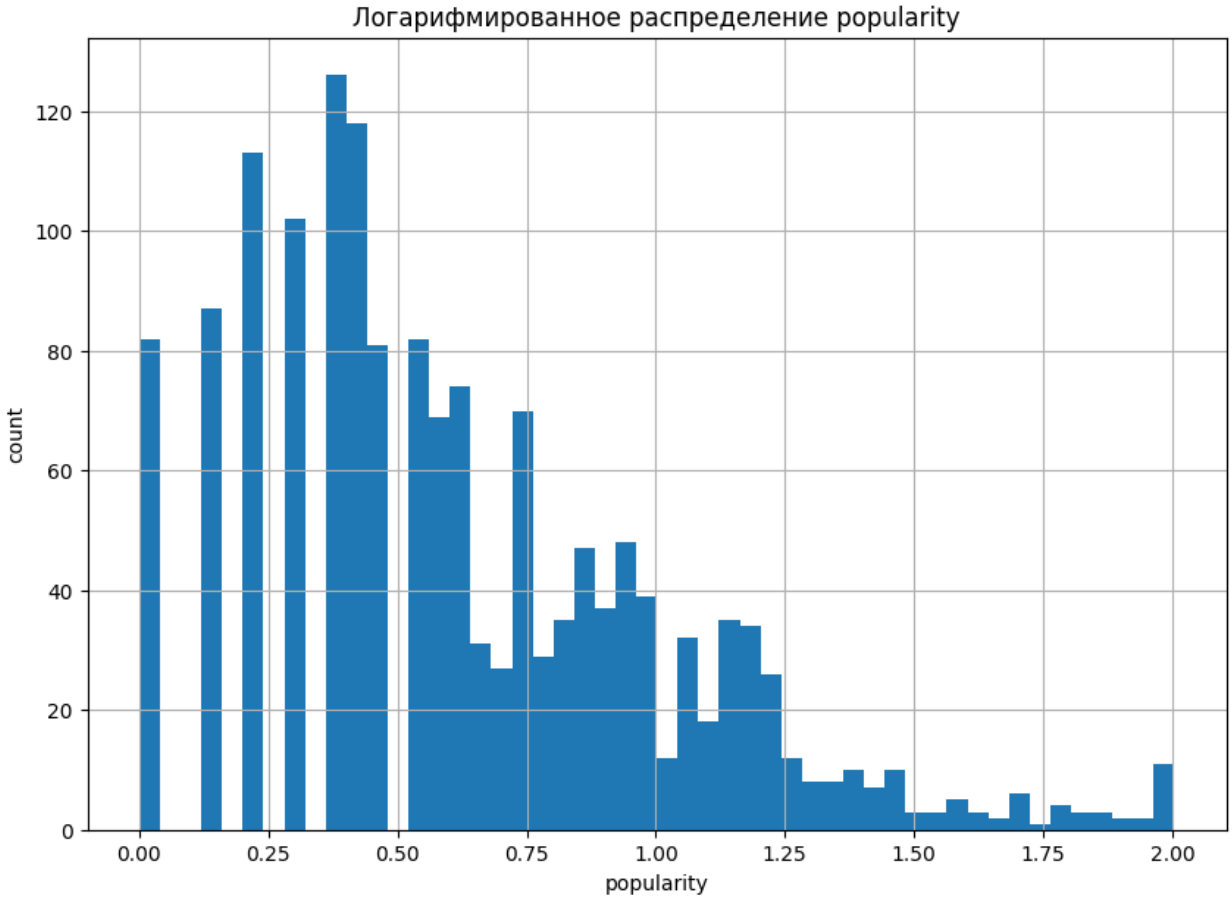
```
import numpy as np

M = 100

data['popularity_log'] = np.log10(data['popularity'] * M + 1)

plt.figure(figsize=(10, 7))
plt.hist(x=data['popularity_log'], bins=50)
plt.xlabel('popularity')
plt.ylabel('count')
plt.title('Логарифмированное распределение popularity')
plt.grid(True)

plt.show();
```



В качестве альтернативного подхода вы можете попробовать дискретизацию признака popularity на основе квантилей (quantile binning), используя функцию qcut() (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.qcut.html). В этом случае вы преобразуете числовой признак popularity в категориальный, для которого в дальнейшем надо будет применить one-hot кодирование.

```
# Признак popularity разобьем на несколько частей
_, bins = pd.qcut(data['popularity'], q=10, retbins=True,
duplicates='drop') # q=10 выбрал экспериментально

data['popularity_qcut'] = pd.cut(data['popularity'], bins=bins,
labels=False, include_lowest=True)

data[['popularity', 'popularity_log', 'popularity_qcut']].sample(10)
```

S.No	popularity	popularity_log	popularity_qcut
560	0.026756	0.565327	5
1076	0.050167	0.779360	6
1148	0.036789	0.670147	6
744	0.006689	0.222429	1
1230	0.030100	0.603148	5
45	0.013378	0.368806	2
1356	0.063545	0.866554	7
439	0.765886	1.889798	9
396	0.010033	0.301756	1
1388	0.040134	0.700130	6

Теперь давайте обработаем **признак numDeadRelations**. Посмотрите на частотное распределение этого признака. Лишь для малого числа персонажей `numDeadRelations>0`.

Создайте признак `boolDeadRelations`. Давайте упростим признак `numDeadRelations`, и просто поделим людей на тех, у кого были хоть какие то отношения с мертвыми персонажами, т.е. `numDeadRelations > 0`, и те, у которых не было, т.е. `numDeadRelations = 0`.

```
data['boolDeadRelations'] = (data['numDeadRelations'] > 0).astype(int)

data[['numDeadRelations', 'boolDeadRelations']].sample(10,
random_state=4) # random_state=4 - для просмотра отработки нашего
преобразования
```

S.No	numDeadRelations	boolDeadRelations
551	0	0
1468	1	1
1404	0	0
1190	0	0
1499	0	0
32	0	0
1335	0	0
1481	0	0
842	1	1
1503	2	1

Наконец, давайте посмотрим на **признак age**. В нем очень много пропущенных значений. Для того, чтобы использовать в модели информацию о возрасте персонажа, мы создадим два новых признака: `age_value` и `age_no_data`

- Там где возраст указан, age_value принимает значение `age`, а `age_no_data` - значение 0.
- Там где возраст не указан, `age_value` принимает значение 0, а `age_no_data` - значение 1.

Фактически, в переменной `age` мы заменяем NaN на 0, но одновременно добавляем в модель еще один бинарный признак `age_no_data`, несущий информацию о том, у каких персонажей не был указан возраст:

```
data['age_value'] = data['age'].fillna(0)
data['age_no_data'] = [1 if np.isnan(x) else 0 for x in data['age']]

data[['age', 'age_value', 'age_no_data']].sample(10, random_state=0)
```

S.No	age	age_value	age_no_data
320	NaN	0.0	1
1265	NaN	0.0	1
799	NaN	0.0	1
580	NaN	0.0	1
415	NaN	0.0	1
495	NaN	0.0	1
619	10.0	10.0	0
1152	43.0	43.0	0
1258	NaN	0.0	1
531	NaN	0.0	1

Этот способ чем-то похож на работу с категориальной переменными с пропущенными значениями, когда мы добавляем еще одну категорию no_data и заменяем NaN на значение этой категории.

Если вы заходите похожим образом образом обработать признак `dateOfBirth`, **обратите внимание**, что у одних и тех же персонажей не указан и возраст, и год рождения. То есть созданные признаки `age_no_data` и `dateOfBirth_no_data` будут полностью совпадать, и в модель надо будет включать только один из признаков: или `age_no_data`, или `dateOfBirth_no_data`.

Задание 1.5. Категориальные признаки с большим количеством категорий

Признак culture содержит информацию о принадлежности к одному из народов во вселенной Игры Престолов.

Давайте посмотрим, какие значения принимает данный признак. По умолчанию метод `value_counts()` игнорирует пропуски в данных, поэтому используем этот метод с параметром **dropna** со значением **False** (см. https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.value_counts.html)

```
data['culture'].value_counts(dropna=False)
```

```
culture
NaN                          1069
Northmen                       94
Ironborn                       91
Free Folk                      45
Braavosi                       39
Valyrian                       28
Dornish                        17
Dothraki                       17
Ghiscari                       17
Reach                          13
Vale mountain clans            12
Rivermen                       10
Valemen                        10
northmen                        9
Westerman                       8
Free folk                       7
Tyroshi                         6
Qartheen                        5
Summer Isles                    4
Astapori                        4
Stormlands                      4
Ironmen                         3
Westeros                        3
Westermen                       3
Dornishmen                      3
Northern mountain clans         3
Lysene                          2
westermen                       2
First Men                       2
Meereenese                      2
Myrish                          2
Sistermen                       2
Crannogmen                      2
Lhazarene                       1
Qarth                           1
Lyseni                          1
Qohor                           1
Norvoshi                        1
Andal                           1
ironborn                        1
Ghiscaricari                    1
Braavos                         1
Lhazareen                       1
Andals                          1
Vale                            1
Wildling                        1
Dorne                           1
```

```
Pentoshi                        1
free folk                       1
Summer Islander                 1
Westerlands                     1
Summer Islands                  1
Name: count, dtype: int64
```

```
# Количество уникальных значений

data['culture'].nunique()
```

```
51
```

Из полученного частотного распределения видно, что для большого числа персонажей значения данного признака не указаны. Также есть много редких значений признака, которые в выборке повторяются один или несколько раз. Причина отчасти в том, что один и тот же народ упоминается в нашем датасете под разными названиями.

Данную проблему мы попытаемся решить, сгруппировав народы в более крупные категории. Так мы одновременно решим проблему того, что один и тот же народ назван в выборке разными способами.

Предоженный вариант группировки имеет определенную логику. Выделяются следующие группы:

- старые нации, которые уже не сущевали как отдельные народы на момент повествования основной линии повествования романов, но отдельные потомки могли еще быть живы
- народы, проживающие в королевствах континента Весторос (для каждого королевства - своя группа)
- народы континента Эссос
- прочие народы

```
cultures_grouped = {
    'Old Nations': ['valyrian', 'first men', 'andal', 'andals',
'rhoynar'],
    'the North': ['northmen', 'northern mountain clans',
'crannogmen'],
    'the Iron Islands': ['ironborn', 'ironborn', 'ironmen'],
    'the Mountain and the Vale': ['valemen', 'vale', 'vale mountain
clans',
                                  'sistermen'],
    'the Isles and Rivers': ['riverlands', 'rivermen'],
    'the Rock': ['westerman', 'westermen', 'westerlands'],
    'the Stormlands': ['stormlander', 'stormlands'],
    'the Reach': ['reach', 'reachmen', 'the reach'],
    'Dorne': ['dornish', 'dornishmen', 'dorne'],
    'Essos Nations': ['astapor', 'astapori', 'braavosi', 'braavos',
'tyroshi', 'lysene', 'lyseni',
                      'myrish', 'pentoshi', 'qartheen', 'qarth',
```

```
'dothraki',
                      'lhazarene', 'lhazareen','meereen',
'meereenese',
                      'norvoshi', 'qohor', 'summer isles', 'summer
islands',
                      'summer islander', 'asshai', "asshai'i",
'norvos', 'ghiscari',
                      'ghiscaricari'],
    'Other Nations': ['ibbenese', 'westeros', 'free folk', 'wildling',
'wildlings', 'naathi']}
```

Обратите внимание, что некоторые варианты названий народов встречаются только в тестовых данных, и не встречаются в обучающих данных. Такая ситуация нередко случается на практике. Поэтому, после обработки обучающих данных и обучения модели важно задать для модели правило, как она должна обрабатывать "незнакомые" категории в категориальных признаках. Например, можно относить объекты с "незнакомой" категорией к некоторой существующей категории или указать формулу расчета для "незнакомой" категории.

Предложенный вам словарь `cultures_grouped` составлен по всем значениям признака `culture`, встречающимся в тренировочном либо в тестовом датасете. Здесь важно, что все укрупненные категории (ключи/keys словаря `cultures_grouped`) представлены в обоих датасетах, а уникальные для тестового датасета названия народов - это отдельные названия народов внутри укрупненных категорий (значения/values словаря). Поэтому, когда вы будете работать с тестовыми - просто применяйте этот словарь без указания правила обработки "незнакомых" категорий.

Давайте приступим к кодировке значений признака culture. Для этого сначала инвертируем словарь *cultures_grouped*

```
# Довольно просто инвертировать словарь, где ключу соответствует одно
значение
# В нашем случае ключу соответствует список значений.
# Ниже показан пример, как можно инвертировать такой словарь

d = {'A': ['a1', 'a2', 'a3'],
     'B': ['b1', 'b2', 'b3', 'b4']}

d_inverted = {}
for k in d.keys():
  for v in d[k]:
      d_inverted.update({v:k})

d_inverted
```

```
{'a1': 'A', 'a2': 'A', 'a3': 'A', 'b1': 'B', 'b2': 'B', 'b3': 'B',
'b4': 'B'}
```

```
# По аналогии с примером выше инвертируйте словарь cultures_grouped
cultures_grouped_inverted = {}
```

```
for group, values in cultures_grouped.items():
    for val in values:
        cultures_grouped_inverted[val] = group

cultures_grouped_inverted
```

```
{'valyrian': 'Old Nations',
 'first men': 'Old Nations',
 'andal': 'Old Nations',
 'andals': 'Old Nations',
 'rhoynar': 'Old Nations',
 'northmen': 'the North',
 'northern mountain clans': 'the North',
 'crannogmen': 'the North',
 'ironborn': 'the Iron Islands',
 'ironmen': 'the Iron Islands',
 'valemen': 'the Mountain and the Vale',
 'vale': 'the Mountain and the Vale',
 'vale mountain clans': 'the Mountain and the Vale',
 'sistermen': 'the Mountain and the Vale',
 'riverlands': 'the Isles and Rivers',
 'rivermen': 'the Isles and Rivers',
 'westerman': 'the Rock',
 'westermen': 'the Rock',
 'westerlands': 'the Rock',
 'stormlander': 'the Stormlands',
 'stormlands': 'the Stormlands',
 'reach': 'the Reach',
 'reachmen': 'the Reach',
 'the reach': 'the Reach',
 'dornish': 'Dorne',
 'dornishmen': 'Dorne',
 'dorne': 'Dorne',
 'astapor': 'Essos Nations',
 'astapori': 'Essos Nations',
 'braavosi': 'Essos Nations',
 'braavos': 'Essos Nations',
 'tyroshi': 'Essos Nations',
 'lysene': 'Essos Nations',
 'lyseni': 'Essos Nations',
 'myrish': 'Essos Nations',
 'pentoshi': 'Essos Nations',
 'qartheen': 'Essos Nations',
 'qarth': 'Essos Nations',
 'dothraki': 'Essos Nations',
 'lhazarene': 'Essos Nations',
 'lhazareen': 'Essos Nations',
 'meereen': 'Essos Nations',
 'meereenese': 'Essos Nations',
 'norvoshi': 'Essos Nations',
```

```
 'qohor': 'Essos Nations',
 'summer isles': 'Essos Nations',
 'summer islands': 'Essos Nations',
 'summer islander': 'Essos Nations',
 'asshai': 'Essos Nations',
 "asshai'i": 'Essos Nations',
 'norvos': 'Essos Nations',
 'ghiscari': 'Essos Nations',
 'ghiscaricari': 'Essos Nations',
 'ibbenese': 'Other Nations',
 'westeros': 'Other Nations',
 'free folk': 'Other Nations',
 'wildling': 'Other Nations',
 'wildlings': 'Other Nations',
 'naathi': 'Other Nations'}
```

Теперь создадим новый столбец с укрупненными значениями culture.

Для этого будем использовать метод `map()` с инвертированным словарем в качестве аргумента (https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.map.html)

Обратите внимание, что в словаре названия народов указаны в нижнем регистре. А в датасете используется как нижний, так и верхний регистр. Поэтому перед применением метода `map()` переведем значения столбца culture в нижний регистр при помощи метода `str.lower()` (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Series.str.lower.html).

```
data['culture_grouped'] =
data['culture'].str.lower().map(cultures_grouped_inverted)
```

Осталось заменить все NaN в созданном столбце на категорию `culture_no_data`:

```
data['culture_grouped'] =
data['culture_grouped'].fillna('culture_no_data')

data['culture_grouped'].value_counts(dropna=False)
```

```
culture_grouped
culture_no_data               1069
Essos Nations                  109
the North                      108
the Iron Islands                95
Other Nations                   57
Old Nations                     32
the Mountain and the Vale       25
Dorne                           21
the Rock                        14
the Reach                       13
the Isles and Rivers            10
```

```
the Stormlands                    4
Name: count, dtype: int64
```

```
data['culture_grouped'].nunique()
```

```
12
```

Распределение сгруппированной переменной выглядит гораздо лучше. Но по прежнему есть несколько слабо представленых групп.

Дальнейшую работу с этим признаком проводите на свое усмотрение для повышения качества прогноза модели. Например, можно объединить несколько слабо представленных категорий в одну или применить другой подход.

Создадим столбец с данными из столбца culture_grouped в котором несколько слабо представленых групп объединены в одну группу 'culture_rare'

```
culture_counts = data['culture_grouped'].value_counts()
rare_cultures = culture_counts[culture_counts < 15].index  # менее 15 представителей

data['culture_grouped_truncated'] = np.where(
    data['culture_grouped'].isin(rare_cultures),
    'culture_rare',
    data['culture_grouped']
)

data['culture_grouped_truncated'].value_counts()
```

```
culture_grouped_truncated
culture_no_data              1069
Essos Nations                 109
the North                     108
the Iron Islands               95
Other Nations                  57
culture_rare                   41
Old Nations                    32
the Mountain and the Vale      25
Dorne                          21
Name: count, dtype: int64
```

Посмотрим есть ли еще категориальные колонки с большим количествоом уникальных значений

```
data.select_dtypes(exclude='number').nunique()
```

```
name                         1557
title                         195
culture                        51
mother                         16
```

```
father                          19
heir                            20
house                          315
spouse                         186
culture_grouped                 12
culture_grouped_truncated        9
dtype: int64
```

У признака house много уникальных значений. Обработаем значения признака аналогично обработке признака culture.

```
data['house'].isna().sum()
```

```
381
```

Но для начала обработаем пропущенные значения

```
# Посмотрим на строки с отсутвующими значениями в колонке 'house'

# data[(data['house'].isna())].head(50)
data[(data['house'].isna())].head()
```

```
                       name title  male culture  dateOfBirth  \
S.No
1     Viserys II Targaryen   NaN     1     NaN          NaN
6         Tommen Baratheon   NaN     1     NaN          NaN
8      Viserys I Targaryen   NaN     1     NaN          NaN
9                  Wilbert   Ser     1     NaN          NaN
12           Will (orphan)   NaN     0     NaN          NaN

                    mother            father                 heir house
spouse  \
S.No

1     Rhaenyra Targaryen  Daemon Targaryen  Aegon IV Targaryen   NaN
NaN
6       Cersei Lannister  Robert Baratheon  Myrcella Baratheon   NaN
NaN
8       Alyssa Targaryen  Baelon Targaryen  Rhaenyra Targaryen   NaN
NaN
9                    NaN               NaN                 NaN   NaN
NaN
12                   NaN               NaN                 NaN   NaN
NaN

      ...  numDeadRelations  popularity  isAlive  popularity_log  \
S.No  ...
1     ...                11    0.605351        0        1.789123
6     ...                 5    1.000000        1        2.004321
```

```
8     ...                  5    0.678930        0        1.838175
9     ...                  0    0.006689        0        0.222429
12    ...                  0    0.003344        1        0.125302

      popularity_qcut  boolDeadRelations  age_value  age_no_data  \
S.No
1                   9                  1        0.0            1
6                   9                  1        0.0            1
8                   9                  1        0.0            1
9                   1                  0        0.0            1
12                  0                  0        0.0            1

      culture_grouped  culture_grouped_truncated
S.No
1     culture_no_data            culture_no_data
6     culture_no_data            culture_no_data
8     culture_no_data            culture_no_data
9     culture_no_data            culture_no_data
12    culture_no_data            culture_no_data

[5 rows x 32 columns]
```

Можно заметить, что в некоторых строках заполнены поля 'father' и/или 'heir' и/или 'spouse и/или 'mother'. Заполним отсутствующие значения в колонке 'house' значениями из колонок 'father' или 'heir' или 'spouse' или 'mother.

```
# Посмотрим как представлены значения в выбранных колонках

data['father'].unique()
# data['heir'].unique()
# data['spouse'].unique()
# data['mother'].unique()

<ArrowStringArray>
[      'Daemon Targaryen',                      nan,       'Robert
Baratheon',
       'Baelon Targaryen',      'Aegon I Targaryen',           'Eddard
Stark',
      'Steffon Baratheon',      'Tristifer IV Mudd',       'Aerion
Targaryen',
    'Viserys I Targaryen',    'Daeron II Targaryen', 'Jaehaerys II
Targaryen',
          'Brandon Stark',    'Aegon III Targaryen',     'Aegon IV
Targaryen',
        'Quellon Greyjoy',          'Qhorwyn Hoare',      'Aegon V
Targaryen',
      'Aenys I Targaryen',          'Halleck Hoare']
Length: 20, dtype: str
```

```
# Извлекаем последнее слово (фамилию) в значениях колонки 'father' /
'heir' / 'spouse' / 'mother и возвращаем готовое значение для колонки
'house'

def extract_surename(value):
    if pd.isna(value):
        return None
    return 'House ' + value.strip().split()[-1]

house_with_missing = data['house'].isna() # только строки с
пропущенными значениями
father_house = data.loc[house_with_missing,
'father'].apply(extract_surename) # фамилии по отцу
heir_house = data.loc[house_with_missing,
'heir'].apply(extract_surename) # фамилии по наследникам
spouse_house = data.loc[house_with_missing,
'spouse'].apply(extract_surename) # фамилии по супругу/супруге
mother_house = data.loc[house_with_missing,
'mother'].apply(extract_surename) # фамилии по матери

# Заполняем пропущенные значения по приоритету father-heir-spouse-
mother-no_data

data.loc[house_with_missing, 'house'] = data.loc[house_with_missing,
'house'].\
    fillna(father_house).\
        fillna(heir_house).\
            fillna(spouse_house).\
                fillna(mother_house). \
                    fillna('house_no_data')

house_counts = data['house'].value_counts(dropna=False)
house_counts
```

```
house
house_no_data                  345
House Frey                      89
Night's Watch                   88
House Stark                     58
House Targaryen                 54
                               ...
House Harlaw of Grey Garden      1
Summer Islands                   1
House Condon                     1
House Lannister of Casterly Rock 1
House Moore                      1
Name: count, Length: 326, dtype: int64
```

```
# data[(data['house'] == 'house_no_data')]
```

```
# house_counts[house_counts > 5].index.tolist()

house_counts[house_counts > 5]
```

```
house
house_no_data                       345
House Frey                           89
Night's Watch                        88
House Stark                          58
House Targaryen                      54
House Lannister                      36
House Tyrell                         33
House Greyjoy                        31
House Osgrey                         20
Faith of the Seven                   15
House Botley                         12
House Hightower                      12
House Arryn                          11
House Martell                        11
House Crakehall                      10
House Florent                        10
House Baratheon                       9
House Waynwood                        9
House Bracken                         9
House Wylde                           8
House Velaryon                        8
House Brax                            8
House Westerling                      8
Stone Crows                           8
House Tully                           8
House Swyft                           7
House Paege                           7
House Whent                           7
House Plumm                           7
House Seaworth                        7
House of Loraq                        6
House Tallhart                        6
House Drumm                           6
House Bolton                          6
House Baelish                         6
House Beesbury                        6
House Baratheon of Dragonstone        6
House Redwyne                         6
House Swann                           6
House Estermont                       6
House Haigh                           6
Name: count, dtype: int64
```

Объединим малочисленные дома в категорию 'house_rare'. Операция аналогична операции с колонкой culture_grouped.

```
rare_houses = house_counts[house_counts < 15].index # менее 15 представителей

data['house_grouped'] = np.where(
    data['house'].isin(rare_houses),
    'house_rare',
    data['house']
)

data['house_grouped'].value_counts(dropna=False)
```

```
house_grouped
house_rare            788
house_no_data         345
House Frey             89
Night's Watch          88
House Stark            58
House Targaryen        54
House Lannister        36
House Tyrell           33
House Greyjoy          31
House Osgrey           20
Faith of the Seven     15
Name: count, dtype: int64
```

Задание 1.6. Категориальные признаки в линейных моделях

Для включения категориальных признаков в линейную модель их нужно преобразовать в числовые признаки.

Если признак принимает одно из двух возможных значений (например, персонаж "появляется" или "не появляется" в 1-й книге), он напрямую кодируется в бинарный признак ("появляется" -> 1, "не появляется" -> 0). Если признак принимает больше двух значений, его можно преобразовать в несколько бинарных при помощи one-hot преобразования. В некоторых случаях бывает полезно объединить некоторые категории, как мы это поступили с признаком *culture*.

Порядковых признаков у нас в задаче нет, поэтому рассматривать их здесь мы не будем.

Для того, чтобы найти все порядковые признаки, посмотрим на количество уникальных значений, которые встречаются в столбцах. Для столбцов с типом object количество уникальных значений мы выводили при помощи метода `describe()` в задании 1.2.

Чтобы посмотреть количество уникальных значений для всех столбцов, можно воспользоваться методом nunique() (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.nunique.html)

```
# Количество уникальнх значений в каждом столбце
data.nunique().sort_values(ascending=False)
```

```
name                          1557
house                          326
title                          195
spouse                         186
popularity_log                 117
popularity                     117
dateOfBirth                    105
age_value                       72
age                             72
culture                         51
heir                            20
father                          19
mother                          16
numDeadRelations                14
culture_grouped                 12
house_grouped                   11
popularity_qcut                 10
culture_grouped_truncated        9
isAliveMother                    2
book2                            2
male                             2
age_no_data                      2
book1                            2
boolDeadRelations                2
isAlive                          2
isAliveHeir                      2
book3                            2
book4                            2
book5                            2
isNoble                          2
isMarried                        2
isAliveSpouse                    2
isAliveFather                    2
dtype: int64
```

```
# Для числовых столбцов можно вывести в одну таблицу более детальную
статистику, объединив выводы describe() и nunique()
# Код ниже требуется дополнить по аналогии с заданием 1.2.
data.describe(include='number').T[['count', 'min', 'max',
'mean']].assign(N_unique_values = data.nunique())
```

```
                     count   min         max        mean
N_unique_values
male                1557.0   0.0    1.000000    0.590880
2
dateOfBirth          279.0 -25.0  299.000000  247.551971
105
book1               1557.0   0.0    1.000000    0.138728
2
book2               1557.0   0.0    1.000000    0.327553
```

```
2
book3               1557.0   0.0    1.000000    0.431599
2
book4               1557.0   0.0    1.000000    0.562620
2
book5               1557.0   0.0    1.000000    0.330122
2
isAliveMother         18.0   0.0    1.000000    0.666667
2
isAliveFather         22.0   0.0    1.000000    0.227273
2
isAliveHeir           21.0   0.0    1.000000    0.666667
2
isAliveSpouse        200.0   0.0    1.000000    0.790000
2
isMarried           1557.0   0.0    1.000000    0.128452
2
isNoble             1557.0   0.0    1.000000    0.439306
2
age                  279.0   0.0  100.000000   35.290323
72
numDeadRelations    1557.0   0.0   15.000000    0.187540
14
popularity          1557.0   0.0    1.000000    0.062400
117
isAlive             1557.0   0.0    1.000000    0.778420
2
popularity_log      1557.0   0.0    2.004321    0.621287
117
popularity_qcut     1557.0   0.0    9.000000    4.394348
10
boolDeadRelations   1557.0   0.0    1.000000    0.044316
2
age_value           1557.0   0.0  100.000000    6.323699
72
age_no_data         1557.0   0.0    1.000000    0.820809
2
```

Посмотрев на количество уникальных значений мы можем выделить категориальные признаки. Например, `popularity` принимает значения от 0 до 1, но это непрерывный числовой признак. Остальные признаки со значениями от 0 до 1 - принимают только два значения, то есть являются бинарными.

Бинарные признаки без NaN полностью готовы для включения в модель. Некоторые бинарные признаки содержат пропуски, поэтому, если вы захотите включить их в модель, их потребуется обработать.

```
# Создадим новые признаки на основе существующих по принципу работы с колонкой 'age'
```

```
for col in ['isAliveMother', 'isAliveFather', 'isAliveHeir',
'isAliveSpouse']:
    data[col + '_value'] = data[col].fillna(0)
    data[col + '_no_data'] = data[col].isna().astype(int)
```

Для бинарных признаков их связь с зависимой переменной можно прикинуть по таблице корреляций. Для категориальных признаков с количеством значений больше двух (или с двумя значениями и NaN) можно сделать one-hot преобразования и посчитать корреряцию зависимой переменной с набором сгенерированных бинарных признаков. В качестве альтернативного подхода можно использовать сравнение средних значений зависимой переменной для разных категорий исследуемого признака. Чем сильнее различаются средние значения целевой переменной между категориями, тем вероятнее, что данный признак связан с зависимой переменной.

```
# попробуйте провести сравнение средних зависимой пременной isAlive
для признака isAliveSpouse
# одним из приведенных ниже способов:

# data.groupby('НАЗВАНИЕ ПРИЗНАКА', dropna = False)['isAlive'].mean()
# pd.pivot_table(data = data, values = 'isAlive', index = 'НАЗВАНИЕ
ПРИЗНАКА', aggfunc=['mean', 'count'], dropna=False)

display(pd.DataFrame(data.groupby('isAliveSpouse', dropna=False)
['isAlive'].mean()).reset_index())
```

```
   isAliveSpouse   isAlive
0            0.0  0.619048
1            1.0  0.753165
2            NaN  0.786293
```

```
pivot_isAliveSpouse_isAlive = pd.pivot_table(data, values='isAlive',
index='isAliveSpouse', aggfunc=['mean', 'count'],
dropna=False).reset_index()
pivot_isAliveSpouse_isAlive.columns = ['_'.join(col).strip() for col
in pivot_isAliveSpouse_isAlive.columns.values]
display(pivot_isAliveSpouse_isAlive)
```

```
   isAliveSpouse_  mean_isAlive  count_isAlive
0             0.0      0.619048             42
1             1.0      0.753165            158
2             NaN      0.786293           1357
```

Посмотрим на оставшиеся с пропусками колонки

```
cols_with_missing = data.isna().sum()
cols_with_missing[cols_with_missing > 0]
```

```
title              840
culture           1069
```

```
dateOfBirth      1278
mother           1539
father           1535
heir             1536
spouse           1357
isAliveMother    1539
isAliveFather    1535
isAliveHeir      1536
isAliveSpouse    1357
age              1278
dtype: int64
```

Практически все оставшиеся с пропусками колонки мы уже обработали ранее.
Удалять пока их не станем, потому что они еще могут пригодиться для дальнейшего анализа.
Лучше в дальнейшем удалить лишние колонки или же выберать необходимые колонки при формировании датасета,
готового к обработке и дальнейшему обучению.

Задание 1.7. Проанализируйте признаки.

- Обработайте категориальные признаки и переведите их в числа. Можете выбрать любой кодировщик. Не забудьте, что потом аналогичным образом вам надо будет преобразовывать тестовый датасет, используя тот же алгоритм кодирования признаков.
- Проанализируйте количественные признаки. Есть ли корреляция между признаками?

```
data.select_dtypes(exclude='number').nunique()
```

```
name                          1557
title                          195
culture                         51
mother                          16
father                          19
heir                            20
house                          326
spouse                         186
culture_grouped                 12
culture_grouped_truncated        9
house_grouped                   11
dtype: int64
```

Для дальнейшей обработки возьмем категориальные признаки: culture_grouped / culture_grouped_truncated и house_grouped.

```
data.shape
```

```
(1557, 41)
```

```
from sklearn.preprocessing import OneHotEncoder

cat_cols = [
    # 'culture_grouped',
    'culture_grouped_truncated',
    'house_grouped'
]

ohe = OneHotEncoder(handle_unknown='ignore', sparse_output=False)

ohe_encoded = ohe.fit_transform(data[cat_cols])

data_encoded = pd.DataFrame(
    ohe_encoded,
    columns=ohe.get_feature_names_out(cat_cols),
    index=data.index
)

data = pd.concat([data, data_encoded], axis=1)

data.shape
```

```
(1557, 61)
```

Корреляционный анализ

```
# Числовые признаки

data.select_dtypes(include='number').columns
```

```
Index(['male', 'dateOfBirth', 'book1', 'book2', 'book3', 'book4',
'book5',
       'isAliveMother', 'isAliveFather', 'isAliveHeir',
'isAliveSpouse',
       'isMarried', 'isNoble', 'age', 'numDeadRelations',
'popularity',
       'isAlive', 'popularity_log', 'popularity_qcut',
'boolDeadRelations',
       'age_value', 'age_no_data', 'isAliveMother_value',
       'isAliveMother_no_data', 'isAliveFather_value',
'isAliveFather_no_data',
       'isAliveHeir_value', 'isAliveHeir_no_data',
'isAliveSpouse_value',
       'isAliveSpouse_no_data', 'culture_grouped_truncated_Dorne',
       'culture_grouped_truncated_Essos Nations',
       'culture_grouped_truncated_Old Nations',
       'culture_grouped_truncated_Other Nations',
       'culture_grouped_truncated_culture_no_data',
       'culture_grouped_truncated_culture_rare',
       'culture_grouped_truncated_the Iron Islands',
```

```
       'culture_grouped_truncated_the Mountain and the Vale',
       'culture_grouped_truncated_the North',
       'house_grouped_Faith of the Seven', 'house_grouped_House Frey',
       'house_grouped_House Greyjoy', 'house_grouped_House Lannister',
       'house_grouped_House Osgrey', 'house_grouped_House Stark',
       'house_grouped_House Targaryen', 'house_grouped_House Tyrell',
       'house_grouped_Night's Watch', 'house_grouped_house_no_data',
       'house_grouped_house_rare'],
      dtype='str')
```

```
# Полная карта корреляций числовых признаков

df_corr_full = data.drop('isAlive', axis=1).corr(numeric_only=True)

df_corr_full.shape
```

```
(49, 49)
```

```
# df_corr_full

# Корреляция количественных принаков (без бинарных)

num_cols = [
#     'male', 'book1', 'book2', 'book3', 'book4', 'book5',
#     'isAliveMother', 'isAliveFather', 'isAliveHeir',
'isAliveSpouse',
#     'isMarried', 'isNoble', 'numDeadRelations', 'boolDeadRelations',
    'dateOfBirth',
    'popularity', 'popularity_log', 'popularity_qcut',
    'age', 'age_value',
#     'age_no_data',
#     'isAliveMother_value', 'isAliveMother_no_data',
'isAliveFather_value', 'isAliveFather_no_data',
#     'isAliveHeir_value', 'isAliveHeir_no_data',
'isAliveSpouse_value', 'isAliveSpouse_no_data'
]

data[num_cols].head()
```

```
      dateOfBirth  popularity  popularity_log  popularity_qcut
age  \
S.No

1             NaN    0.605351        1.789123                9   NaN

2           208.0    0.896321        1.957282                9  97.0

3             NaN    0.267559        1.443355                9   NaN

4            82.0    0.183946        1.287682                9  23.0
```

```
5           276.0    0.043478        0.728177                 6  29.0
```

```
      age_value
S.No
1           0.0
2          97.0
3           0.0
4          23.0
5          29.0
```

```
df_corr_num = data[[col for col in data.columns if col in num_cols] +
['isAlive']].corr()
df_corr_num
```

```
                dateOfBirth       age  popularity  popularity_log  \
dateOfBirth        1.000000 -0.504566   -0.052242       -0.083420
age               -0.504566  1.000000    0.012400       -0.013953
popularity        -0.052242  0.012400    1.000000        0.778372
popularity_log    -0.083420 -0.013953    0.778372        1.000000
popularity_qcut   -0.047540 -0.055573    0.565121        0.944993
age_value         -0.504566  1.000000    0.207874        0.301053
isAlive            0.655881 -0.311115   -0.194187       -0.242497

                popularity_qcut  age_value   isAlive
dateOfBirth           -0.047540  -0.504566  0.655881
age                   -0.055573   1.000000 -0.311115
popularity             0.565121   0.207874 -0.194187
popularity_log         0.944993   0.301053 -0.242497
popularity_qcut        1.000000   0.290037 -0.231082
age_value              0.290037   1.000000 -0.181938
isAlive               -0.231082  -0.181938  1.000000
```

Построим тепловую карту корреляций количественных признаков

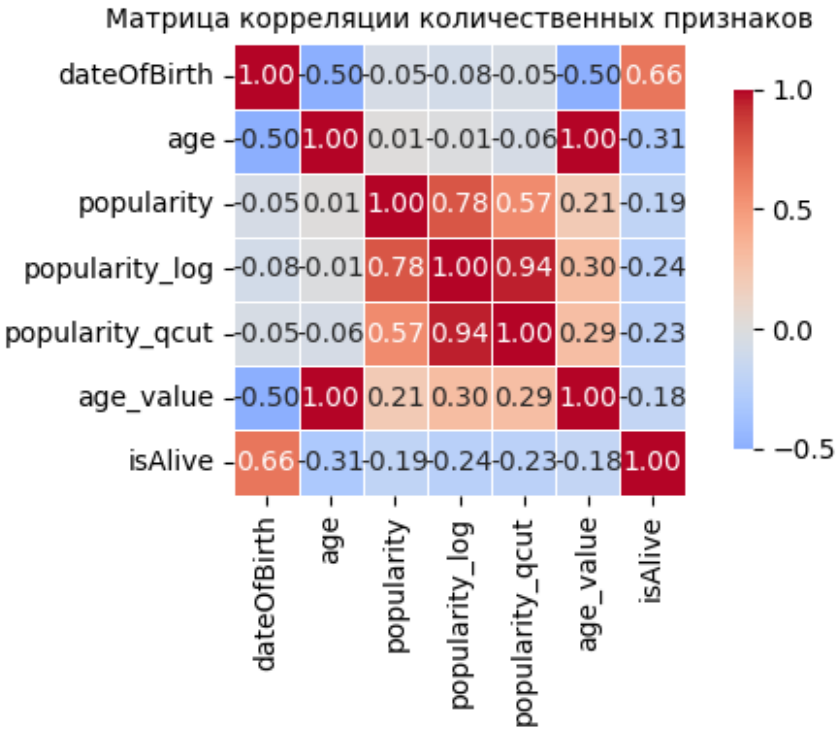
```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(6, 4))

sns.heatmap(
    df_corr_num,
    annot=True,
    fmt='.2f',
    cmap='coolwarm',
    center=0,
    square=True,
    linewidths=0.5,
    cbar_kws={'shrink': 0.8}
)
```

```
plt.title('Матрица корреляции количественных признаков', fontsize=10)
plt.tight_layout()

plt.show();
```




```
data.select_dtypes(include='number').columns
```

```
Index(['male', 'dateOfBirth', 'book1', 'book2', 'book3', 'book4',
'book5',
       'isAliveMother', 'isAliveFather', 'isAliveHeir',
'isAliveSpouse',
       'isMarried', 'isNoble', 'age', 'numDeadRelations',
'popularity',
       'isAlive', 'popularity_log', 'popularity_qcut',
'boolDeadRelations',
       'age_value', 'age_no_data', 'isAliveMother_value',
       'isAliveMother_no_data', 'isAliveFather_value',
'isAliveFather_no_data',
       'isAliveHeir_value', 'isAliveHeir_no_data',
'isAliveSpouse_value',
       'isAliveSpouse_no_data', 'culture_grouped_truncated_Dorne',
       'culture_grouped_truncated_Essos Nations',
       'culture_grouped_truncated_Old Nations',
       'culture_grouped_truncated_Other Nations',
       'culture_grouped_truncated_culture_no_data',
```

```
       'culture_grouped_truncated_culture_rare',
       'culture_grouped_truncated_the Iron Islands',
       'culture_grouped_truncated_the Mountain and the Vale',
       'culture_grouped_truncated_the North',
       'house_grouped_Faith of the Seven', 'house_grouped_House Frey',
       'house_grouped_House Greyjoy', 'house_grouped_House Lannister',
       'house_grouped_House Osgrey', 'house_grouped_House Stark',
       'house_grouped_House Targaryen', 'house_grouped_House Tyrell',
       'house_grouped_Night's Watch', 'house_grouped_house_no_data',
       'house_grouped_house_rare'],
      dtype='str')
```

```
df_corr_num['age'].abs().drop('age').sort_values(ascending=False)
```

```
age_value           1.000000
dateOfBirth         0.504566
isAlive             0.311115
popularity_qcut     0.055573
popularity_log      0.013953
popularity          0.012400
Name: age, dtype: float64
```

```
df_corr_num['popularity'].abs().drop('popularity').sort_values(ascendi
ng=False)
```

```
popularity_log      0.778372
popularity_qcut     0.565121
age_value           0.207874
isAlive             0.194187
dateOfBirth         0.052242
age                 0.012400
Name: popularity, dtype: float64
```

Устраним мультиколлинеарность.
Для дальнейшего анализа оставим признаки 'popularity_log', 'popularity_qcut', 'age_value'.
Между признаками 'popularity_log', 'popularity_qcut' будет необходимо тоже сделать выбор.

```
# Корреляция числовых принаков (с бинарными)

num_cols_full = [
    'male', 'book1', 'book2', 'book3', 'book4', 'book5',
    # 'isAliveMother', 'isAliveFather', 'isAliveHeir',
'isAliveSpouse',
    'isMarried', 'isNoble', 'numDeadRelations', 'boolDeadRelations',
    # 'dateOfBirth',
    # 'popularity',
    'popularity_log',
    'popularity_qcut',
    # 'age',
    'age_value',
```

```
    'age_no_data',
    'isAliveMother_value', 'isAliveMother_no_data',
'isAliveFather_value', 'isAliveFather_no_data',
    'isAliveHeir_value', 'isAliveHeir_no_data', 'isAliveSpouse_value',
'isAliveSpouse_no_data'
]

df_corr_num_full = data[[col for col in data.columns if col in
num_cols_full]].corr()
```

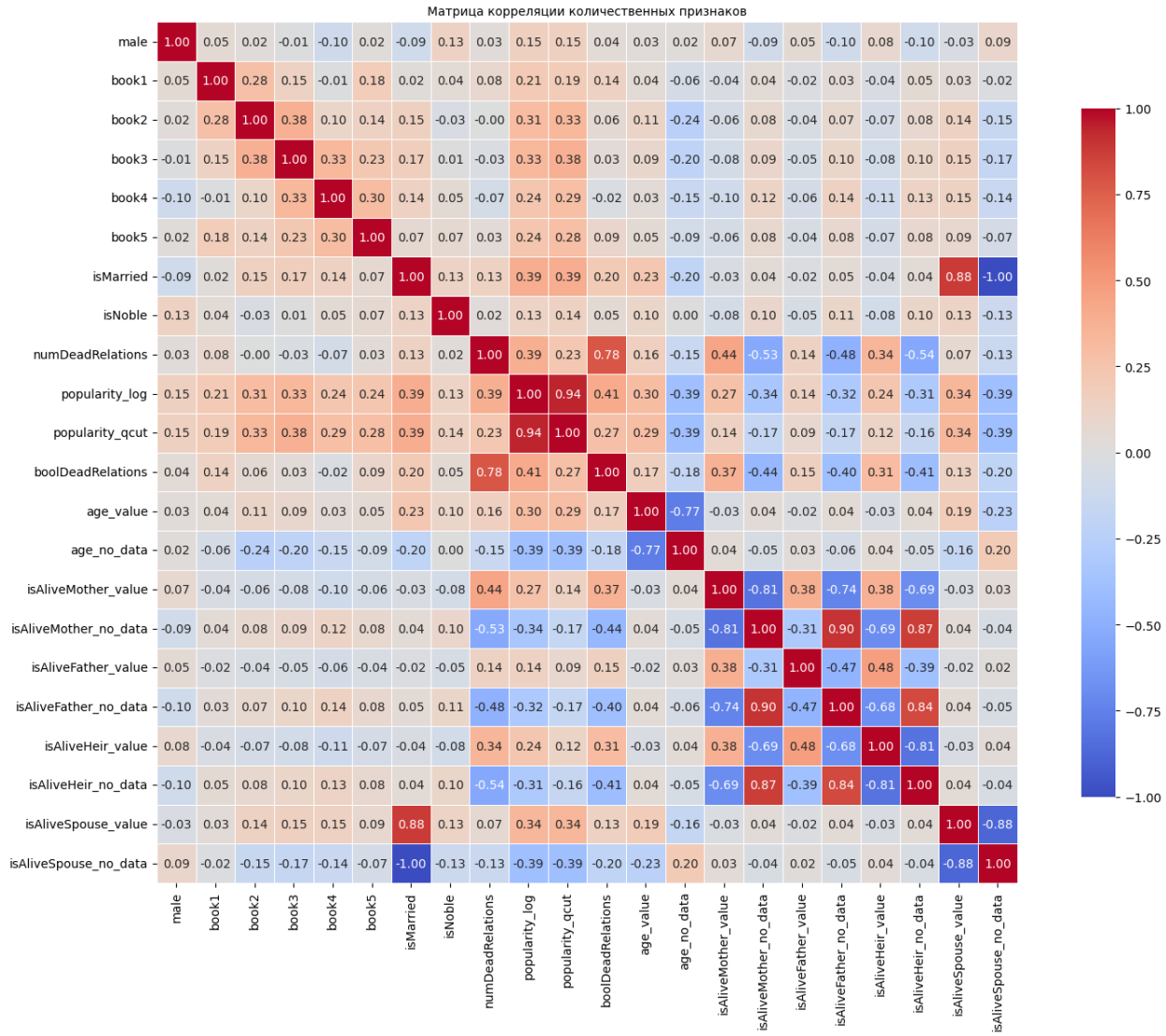
Построим тепловую карту корреляций количественных признаков

```
plt.figure(figsize=(16, 12))

sns.heatmap(
    df_corr_num_full,
    annot=True,
    fmt='.2f',
    cmap='coolwarm',
    center=0,
    square=True,
    linewidths=0.5,
    cbar_kws={'shrink': 0.8}
)

plt.title('Матрица корреляции количественных признаков', fontsize=10)
plt.tight_layout()

plt.show();
```



Наблюдается высокая корреляция между несколькими признаками.
isMarried и isAliveSpouse_no_data - полная отрицательная корреляция
isMarried и isAliveSpouse_value - сильная корреляция
popularity_log и popularity_qcut - сильная корреляция (по факту один и тот же признак)
numDeadRelations и boolDeadRelations - сильная корреляция (очень схожие признаки)

Выберем признаки для анализа их влияния на целевую переменную.

```
selected_cols_corr = [
    'male', 'book1', 'book2', 'book3', 'book4', 'book5',
    'isMarried',
    # 'isAliveSpouse_value', 'isAliveSpouse_no_data',
    'isNoble',
    # 'popularity_log',
    'popularity_qcut',
    # 'numDeadRelations',
```

```
    'boolDeadRelations',
    'age_value',
    'age_no_data',
    'isAliveMother_value', 'isAliveMother_no_data',
'isAliveFather_value', 'isAliveFather_no_data',
    'isAliveHeir_value', 'isAliveHeir_no_data',
]
```

Построим тепловую карту корреляций признаков

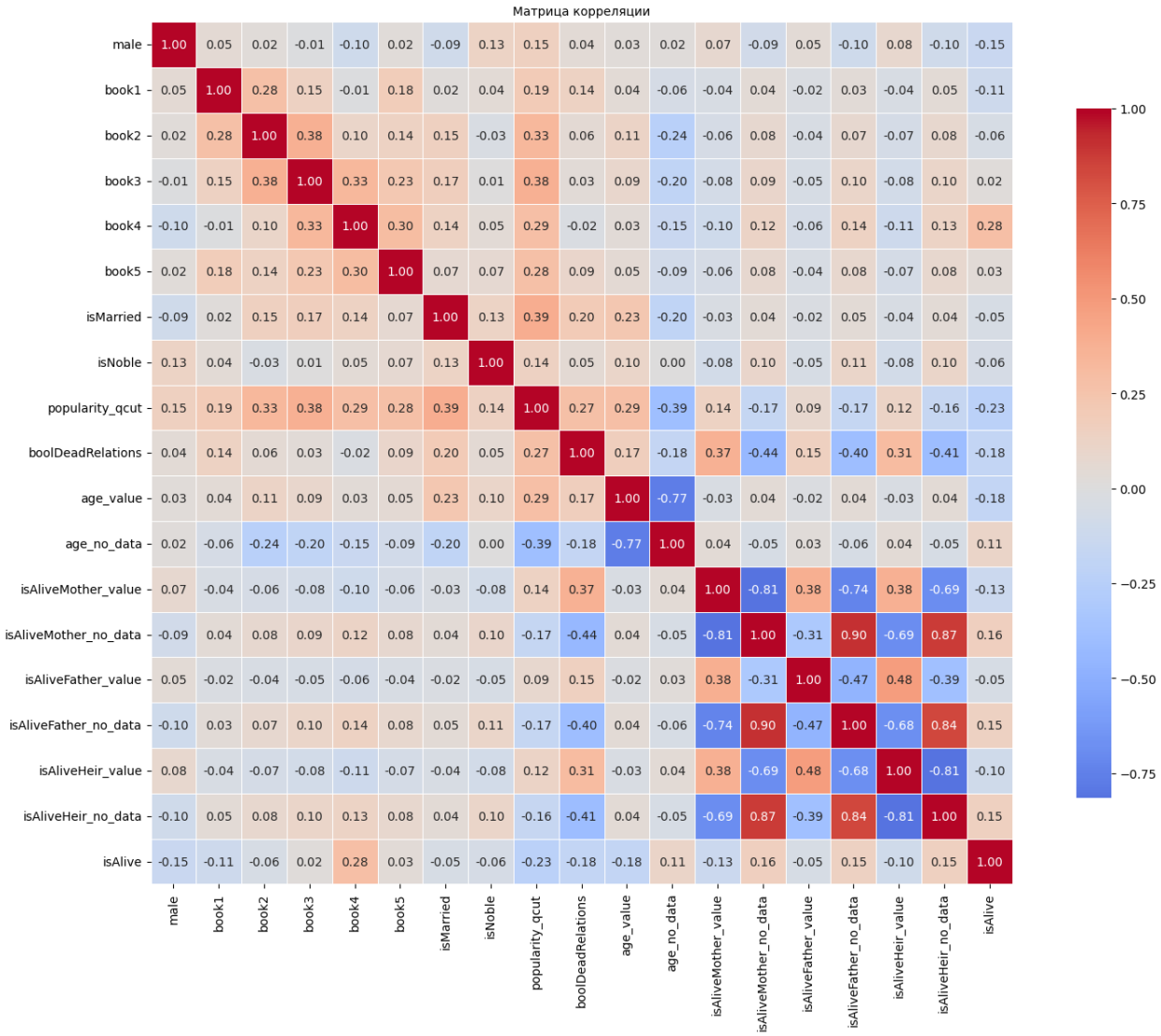
```
df_corr_target = data[[col for col in data.columns if col in
selected_cols_corr] + ['isAlive']].corr()

plt.figure(figsize=(16, 12))

sns.heatmap(
    df_corr_target,
    annot=True,
    fmt='.2f',
    cmap='coolwarm',
    center=0,
    square=True,
    linewidths=0.5,
    cbar_kws={'shrink': 0.8}
)

plt.title('Матрица корреляции', fontsize=10)
plt.tight_layout()

plt.show();
```



Задание 1.8. Проанализируйте влияние признаков на целевую переменную.

```
# df_corr_target

df_corr_target['isAlive']
```

```
male                  -0.148290
book1                 -0.112468
book2                 -0.055998
book3                  0.018427
book4                  0.284014
book5                  0.032531
isMarried             -0.049381
isNoble               -0.060572
popularity_qcut       -0.231082
boolDeadRelations     -0.178177
age_value             -0.181938
```

```
age_no_data               0.113629
isAliveMother_value      -0.129817
isAliveMother_no_data     0.159302
isAliveFather_value      -0.051718
isAliveFather_no_data     0.145771
isAliveHeir_value        -0.096622
isAliveHeir_no_data       0.152124
isAlive                   1.000000
Name: isAlive, dtype: float64
```

Отсортируем признаки по абсолютному значению влияния на целевую переменную.

```
corr_target =
df_corr_target['isAlive'].abs().drop('isAlive').sort_values(ascending=
False)
pd.DataFrame(corr_target).round(6)
```

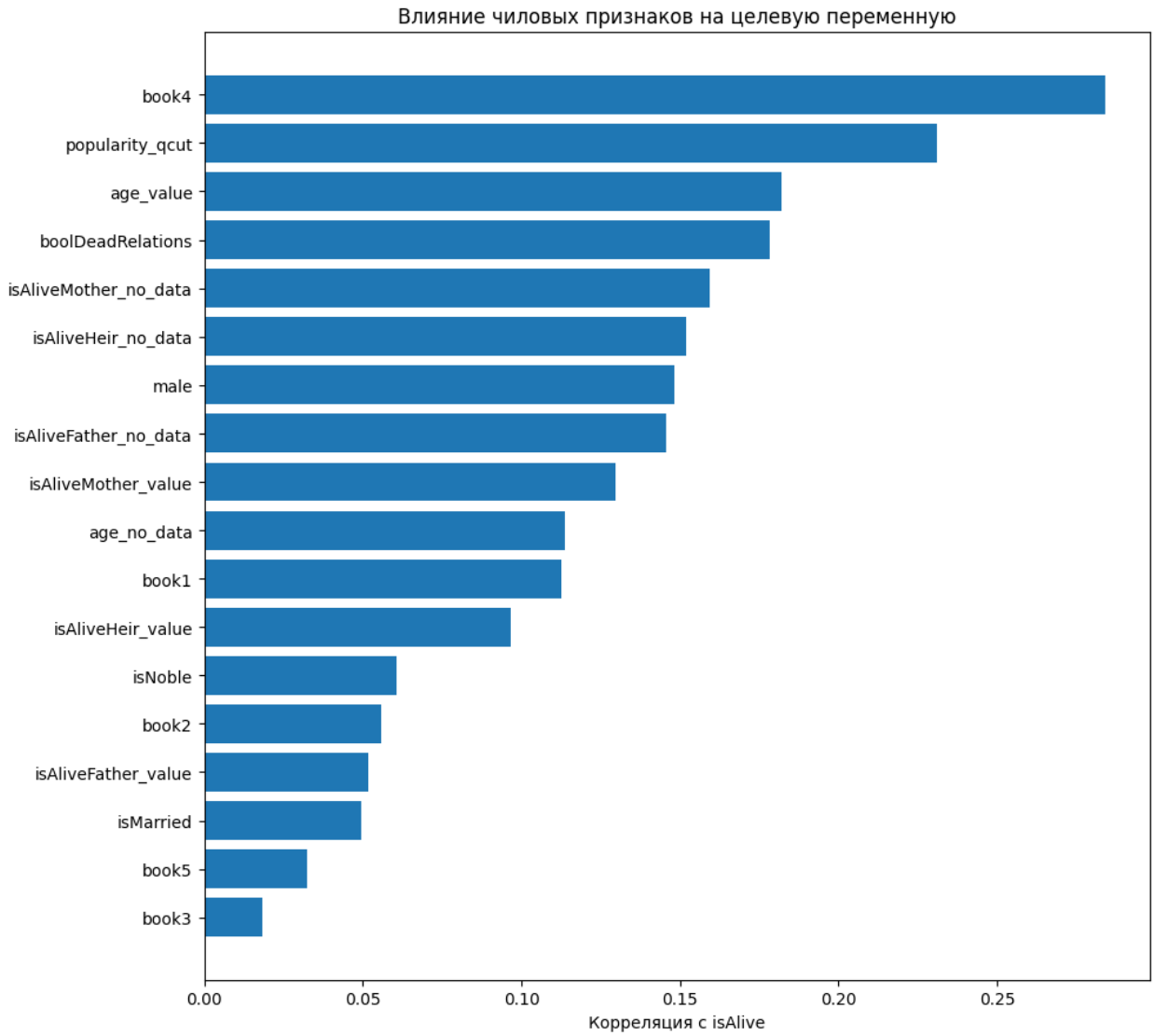
	isAlive
book4	0.284014
popularity_qcut	0.231082
age_value	0.181938
boolDeadRelations	0.178177
isAliveMother_no_data	0.159302
isAliveHeir_no_data	0.152124
male	0.148290
isAliveFather_no_data	0.145771
isAliveMother_value	0.129817
age_no_data	0.113629
book1	0.112468
isAliveHeir_value	0.096622
isNoble	0.060572
book2	0.055998
isAliveFather_value	0.051718
isMarried	0.049381
book5	0.032531
book3	0.018427

```
plt.figure(figsize=(10, 9))
plt.barh(corr_target.index, corr_target.values)

plt.xlabel('Корреляция с isAlive')
plt.title('Влияние чиловых признаков на целевую переменную')

plt.gca().invert_yaxis()
plt.tight_layout()

plt.show();
```



Отбор признаков

```
cols_with_missing = data.isna().sum()
cols_with_missing[cols_with_missing > 0]
```

```
title             840
culture          1069
dateOfBirth      1278
mother           1539
father           1535
heir             1536
spouse           1357
isAliveMother    1539
isAliveFather    1535
isAliveHeir      1536
isAliveSpouse    1357
```

```
age                1278
dtype: int64
```

```
print(cols_with_missing[cols_with_missing > 0].index.tolist())
```

```
['title', 'culture', 'dateOfBirth', 'mother', 'father', 'heir',
'spouse', 'isAliveMother', 'isAliveFather', 'isAliveHeir',
'isAliveSpouse', 'age']
```

С пропущенными значениями остались только признаки, ранее уже обработанные или явдяющиеся идентификаторами.
Данные признаки в финальный датасет не включаем.

Выбор признаков для исключения из финального датасета:

```
# Некоторые признаки для исключения выбраны по итогам экспериментов

cols_to_drop = [
    'name', 'house', 'culture_grouped', 'culture_grouped_truncated',
'house_grouped',
    'title', 'culture', 'dateOfBirth', 'mother', 'father', 'heir',
'spouse',
    'isAliveMother', 'isAliveFather', 'isAliveHeir', 'isAliveSpouse',
'age',
    # 'male', 'book1', 'book2', 'book3', 'book4', 'book5',
    # 'isNoble',
    'popularity_log',
    # 'popularity_qcut',
    'numDeadRelations',
    # 'boolDeadRelations',
    # 'age_value',
    # 'age_no_data',

    'isMarried',

    # 'isAliveSpouse_value',
    # 'isAliveMother_value',
    # 'isAliveFather_value',
    # 'isAliveHeir_value',

    'isAliveSpouse_no_data',
    # 'isAliveMother_no_data',
    'isAliveFather_no_data',
    'isAliveHeir_no_data',
]

data_final = data.drop([col for col in data.columns if col in
cols_to_drop], axis=1)
```

Задание 1.9. Создайте переменные `X`, которая будет хранить только значения признаков, которые вы отобрали для включения в модель, и `y`, которая будет хранить только значения целевой переменной.

```
X = data_final.drop('isAlive', axis=1).copy()
y = data_final['isAlive']
```

Задание 1.10. Разделите датасет обучащую и валидационные части (train и val) при помощи функции `train_test_split` (https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html)

```
from sklearn.model_selection import train_test_split

# Баланс классов в целевой переменной

data_final['isAlive'].value_counts(normalize=True)
```

```
isAlive
1    0.77842
0    0.22158
Name: proportion, dtype: float64
```

Классы в целевой переменной 'isAlive' несбалансировны. Применим stratify при разбиении на тренировочную и тестовую выборки.

```
# не забудьте в функции train_test_split задать параметр random_state,
# чтобы обеспечить повторяемость разбиения выборки на train и
validation части.
# Это позволит сравнивать метрики моделей с различными методами
подготовки признаков

X_train, X_val, y_train, y_val = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    shuffle=True,
    stratify=y
)

len(data_final.select_dtypes(exclude='number').sum()) # проверка на
наличие нечисловых признаков
```

```
0
```

```
# data_final.select_dtypes(exclude='number').head()
```

Линейные модели чувствительны к масштабу, поэтому необходимо масштабировать признаки.

```
from sklearn.preprocessing import StandardScaler

std_scaler = StandardScaler()

X_train_scaled = std_scaler.fit_transform(X_train)
X_val_scaled = std_scaler.transform(X_val)
```

Часть 2. Обучение моделей

В данной части домашнего задания, мы хотим научиться обучать модели для задачи классификации на наших данных.

Задание 2.1.

Вы можете работать с одной из предложенных моделей из библиотеки `sklearn`

- LogisticRegression
- RandomForestClassifier
- AdaBoostClassifier
- GaussianProcessClassifier
- GaussianNB
- KNeighborsClassifier
- SVC
- DecisionTreeClassifier

Однако в этом домашнем задании мы предлагаем выбрать и поработать с моделью `LogisticRegression`.

```
from sklearn.linear_model import LogisticRegression
```

Импортируйте остальные модели из библиотеки `sklearn`. Чтобы понять как это сделать, воспользуйтесь официальный документацией `sklearn` → тык. По ключевому названию модели, вы сможете найти необходимую информацию о том, как можно импортировать модель из библиотеки.

Задание 2.2. Обучите модель и сделайте предсказание на тестовой выборке

В качестве примера, обучим модель `LogisticRegression` и сделаем на ней предсказания на тестовой выборке.

```
# Шаг 1. создание модели
log_model = LogisticRegression(max_iter=1000, random_state=42)

# Шаг 2. обучение модели
log_model.fit(X_train_scaled, y_train)

# Шаг 3. Предсказание на тестовых данных
y_pred = log_model.predict(X_val_scaled)
```

Часть 3. Оцените качество моделей

Вам необходимо познакомиться с метриками задачи классификации из sklearn. Оцените все модели и выберите лучшую по метрике качества Accuracy.

С метриками классификации вы можете ознакомиться в Yandex ML Book.

Для простоты в данном домашнем задании мы будем работать с самой базовой метрикой для задачи классификации - accuracy.

Задание 3.1. Вам необходимо посчитать метрику для всех моделей и выбрать лучшую модель.

Сначала импортируем необходимую функцию из библиотеки sklearn для подсчета accuracy.

```
from sklearn.metrics import accuracy_score
```

В качестве примера, посчитаем метрику accuracy для модели `LogisticRegression`

```
# Шаг 3. Предсказание на тестовых данных

# ваш код здесь

# Шаг 4. Оценка предсказания по метрике accuracy
accuracy = accuracy_score(y_val, y_pred)
print("Accuracy : %.4f" % accuracy)
```

```
Accuracy : 0.7917
```

Возможно, вы решите вернуться на несколько шагов и попробовать другие варианты преобразования и подбора признаков в модель. **Выберите** лучшую модель.

Тестовый датасет

В самом начале нашего домашнего задания мы скачивали тестовый датасет. Загрузите его в Pandas DataFrame при помощи функции read_csv

```
df_test = pd.read_csv(PATH + 'game_of_thrones_test.csv',
index_col='S.No')
```

По аналогии с тем, как мы работали с обучающим датасетом, давайте посмотрим в тестовом датасете на статистики признаков с разными типами данных

```
#Подсказка
df_test.loc[df_test['age']<0] # Находим данные с ошибками в дате
рождения и возрасте
```

```
        name title  male culture  dateOfBirth mother father heir  \
S.No
1685  Doreah   NaN     0  Lysene     278279.0    NaN    NaN  NaN
```

```
1869  Rhaego   NaN     0     NaN     298299.0    NaN    NaN  NaN

                house spouse  ...  book5  isAliveMother  isAliveFather
\
S.No                          ...

1685  House Targaryen    NaN  ...      1            NaN            NaN

1869  House Targaryen    NaN  ...      0            NaN            NaN


      isAliveHeir  isAliveSpouse  isMarried  isNoble       age  \
S.No
1685          NaN            NaN          0        0 -277980.0
1869          NaN            NaN          0        0 -298001.0

      numDeadRelations  popularity
S.No
1685                 0    0.110368
1869                 6    0.167224

[2 rows x 24 columns]
```

```
#Вы можете аккуратно поправить тестовые данные, воспользовавшись,
например следующим кодом:
df_test.loc[1685, 'dateOfBirth'] = 278.0
df_test.loc[1685, 'age'] = 0.0

df_test.loc[1869, 'dateOfBirth'] = 299.0
df_test.loc[1869, 'age'] = 0.0
```

Преобразуйте признаки в тестовом датасете по тому же пайплайну, как вы преобразовывали обучающие данные. Примените вашу лучшую модель на тестовом датасете для получения прогноза целевой переменной `isAlive`

```
M = 100
df_test['popularity_log'] = np.log10(df_test['popularity'] * M + 1)
df_test['popularity_qcut'] = pd.cut(df_test['popularity'], bins=bins,
labels=False, include_lowest=True)

df_test['boolDeadRelations'] = (df_test['numDeadRelations'] >
0).astype(int)

df_test['age_value'] = df_test['age'].fillna(0)
df_test['age_no_data'] = df_test['age'].isna().astype(int)

for col in ['isAliveMother', 'isAliveFather', 'isAliveHeir',
'isAliveSpouse']:
    df_test[col + '_value'] = df_test[col].fillna(0)
    df_test[col + '_no_data'] = df_test[col].isna().astype(int)
```

```
df_test['culture_grouped'] =
df_test['culture'].str.lower().map(cultures_grouped_inverted)
df_test['culture_grouped'] =
df_test['culture_grouped'].fillna('culture_no_data')

df_test['culture_grouped_truncated'] = np.where(
    df_test['culture_grouped'].isin(rare_cultures),
    'culture_rare',
    df_test['culture_grouped']
)

house_with_missing = df_test['house'].isna()
father_house = df_test.loc[house_with_missing,
'father'].apply(extract_surename)
heir_house = df_test.loc[house_with_missing,
'heir'].apply(extract_surename)
spouse_house = df_test.loc[house_with_missing,
'spouse'].apply(extract_surename)
mother_house = df_test.loc[house_with_missing,
'mother'].apply(extract_surename)

df_test.loc[house_with_missing, 'house'] =
df_test.loc[house_with_missing, 'house'].\
    fillna(father_house).\
        fillna(heir_house).\
            fillna(spouse_house).\
                fillna(mother_house).\
                    fillna('house_no_data')

df_test['house_grouped'] = np.where(
    df_test['house'].isin(rare_houses),
    'house_rare',
    df_test['house']
)


ohe_test = ohe.transform(df_test[cat_cols])

df_test_encoded = pd.DataFrame(
    ohe_test,
    columns=ohe.get_feature_names_out(cat_cols),
    index=df_test.index
)

df_test = pd.concat([df_test, df_test_encoded], axis=1)

# Формирование X_test
```

```
X_test = df_test.drop([col for col in df_test.columns if col in
cols_to_drop], axis=1)

# Проверим совпадает ли набор и порядок колонок в тренировочном и
тестовом датасетах

assert list(X_train.columns) == list(X_test.columns)

# Масштабирование
X_test_scaled = std_scaler.transform(X_test)

# Предсказание
y_test_pred = log_model.predict(X_test_scaled)
```

Файл `submission.csv`

Вам нужно вместо значений в `submission.csv` файле в колонке `isAlive`, подставить свои предсказания и сохранить измененный файл.

```
# !gdown 1M14conWjAW2QLoyCXbHEAy8bql2f99eF

FOLDER_SUBS = 'subs'

# Проверка наличия папки и создание по необходимости
os.makedirs(FOLDER_SUBS, exist_ok=True)

# Скачивание submission

FILE_ID = '1M14conWjAW2QLoyCXbHEAy8bql2f99eF'
PATH = os.path.join(FOLDER_SUBS, 'submission.csv')

# Проверка наличия файла и скачивание по необходимости
if not os.path.exists(PATH):
    gdown.download(f'https://drive.google.com/uc?id={FILE_ID}', PATH,
quiet=True)
    print('Файл скачан:', PATH)
else:
    print('Файл уже скачан')
```

```
Файл уже скачан
```

```
submission = pd.read_csv(os.path.join(FOLDER_SUBS, 'submission.csv'),
index_col='S.No')

submission
```

```
       isAlive
S.No
1558         0
1559         0
1560         0
```

```
1561         0
1562         0
...        ...
1942         0
1943         0
1944         0
1945         0
1946         0

[389 rows x 1 columns]
```

Как сохранить измененный Pandas DataFrame в csv файл:

```
# submission.to_csv("/content/new_submission.csv", index=False)

submission['isAlive'] = y_test_pred

submission_name = 'new_submission'
submission.to_csv(os.path.join(FOLDER_SUBS, f'{submission_name}.csv'),
index=False)

# Проверим несколько строк submission

submission_view = pd.read_csv(os.path.join(FOLDER_SUBS,
f'{submission_name}.csv'))

print(submission_view.head())
```

```
   isAlive
0        0
1        1
2        1
3        1
4        1
```

Accuracy вашего решения равен 0.6863753213367609. Это дает вам 5.0 баллов. Если вы хотите больше баллов, можете попробовать еще раз.

Поиск лучшей модели и ее параметров

Обучим еще несколько простых моделей, помимо модели логистической регресии, и добавим перебор гиперпараметров по сетке.

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier

from sklearn.model_selection import GridSearchCV
```

```
# Сетка с перебором моделей и их гиперпараметров
models = {
    'LogisticRegression': {
        'model': LogisticRegression(max_iter=1000, random_state=42),
        'params': {
            'C': [0.1, 1, 10, 100],
            'solver': ['liblinear', 'lbfgs']
        },
    },
    'RandomForest': {
        'model': RandomForestClassifier(random_state=42),
        'params': {
            'n_estimators': [100, 200],
            'max_depth': [5, 10, None],
            'min_samples_split': [2, 5]
        },
    },
    'KNN': {
        'model': KNeighborsClassifier(),
        'params': {
            'n_neighbors': [3, 5, 7, 9],
            'weights': ['uniform', 'distance']
        },
    },
}

best_score = 0
best_model = None
best_params = None
best_name = None

for name, config in models.items():
    gs = GridSearchCV(config['model'], config['params'], cv=5,
scoring='accuracy', n_jobs=-1, verbose=1)
    gs.fit(X_train_scaled, y_train)

    print(f'Модель {name}')
    print(f'Лучшие параметры: {gs.best_params_}')
    print(f'Лучшая точность: {gs.best_score_:.4f}')

    y_pred = gs.predict(X_val_scaled)
    val_acc = accuracy_score(y_val, y_pred)
    print(f'Лучшая точность на валидации:{val_acc:.4f}\n')

    if val_acc > best_score:
        best_score = val_acc
        best_model = gs.best_estimator_
        best_params = gs.best_params_
        best_name = name
```

```
print(f'Лучшая модель: {best_name}')
print(f'Лучшие параметры: {best_params}')
print(f'Лучшая точность на валидации: {best_score:.4f}')

Fitting 5 folds for each of 8 candidates, totalling 40 fits
Модель LogisticRegression
Лучшие параметры: {'C': 1, 'solver': 'liblinear'}
Лучшая точность: 0.8281
Лучшая точность на валидации:0.7917

Fitting 5 folds for each of 12 candidates, totalling 60 fits

Модель RandomForest
Лучшие параметры: {'max_depth': 10, 'min_samples_split': 2,
'n_estimators': 100}
Лучшая точность: 0.8305
Лучшая точность на валидации:0.8141

Fitting 5 folds for each of 8 candidates, totalling 40 fits
Модель KNN
Лучшие параметры: {'n_neighbors': 7, 'weights': 'uniform'}
Лучшая точность: 0.8209
Лучшая точность на валидации:0.8205

Лучшая модель: KNN
Лучшие параметры: {'n_neighbors': 7, 'weights': 'uniform'}
Лучшая точность на валидации: 0.8205

y_test_pred = best_model.predict(X_test_scaled)

submission = pd.read_csv(os.path.join(FOLDER_SUBS, 'submission.csv'),
index_col='S.No')
submission['isAlive'] = y_test_pred
submission_name = 'submission_gs'
submission.to_csv(os.path.join(FOLDER_SUBS, f'{submission_name}.csv'),
index=False)
```

Accuracy вашего решения равен 0.6915167095115681. Это дает вам 5.0 баллов. Если вы хотите больше баллов, можете попробовать еще раз.

С дропом представленных ниже признаков результат модели RandomForest
{'max_depth': None, 'min_samples_split': 5, 'n_estimators': 100}
составил: Accuracy вашего решения равен 0.7429305912596401. Это дает вам 5.0 баллов. Если вы хотите больше баллов, можете попробовать еще раз.

```
# Некоторые признаки для исключения выбраны по итогам экспериментов

cols_to_drop = [
    'name', 'house', 'culture_grouped', 'culture_grouped_truncated',
'house_grouped',
```

```
    'title', 'culture', 'dateOfBirth', 'mother', 'father', 'heir',
'spouse',
    'isAliveMother', 'isAliveFather', 'isAliveHeir', 'isAliveSpouse',
'age',
    # 'male', 'book1', 'book2', 'book3', 'book4', 'book5',
    # 'isNoble',
    # 'popularity_log',
    'popularity_qcut',
    'numDeadRelations',
    # 'boolDeadRelations',
    # 'age_value',
    # 'age_no_data',

    # 'isMarried',

    'isAliveSpouse_value',
    'isAliveMother_value',
    # 'isAliveFather_value',
    'isAliveHeir_value',

    'isAliveSpouse_no_data',
    'isAliveMother_no_data',
    'isAliveFather_no_data',
    'isAliveHeir_no_data',
]
```